

Valkyrie: A Local Provenance Experiment for Unified Security and Privacy Decisions

Research prototype paper

Evidence cutoff: 2026-08-28

Windows endpoint telemetry, local-only correlation, and a narrow synthetic experiment

ABSTRACT

Endpoint security tools typically treat security alerts, network behavior, and privacy controls as separate products. Valkyrie investigates a narrower alternative: whether a local endpoint can connect these observations through a causality graph and make a more explainable, policy-constrained decision about what a process chain caused. The system normalizes process, DNS, network, persistence, ETW, and privacy events; records process ancestry and attached artifacts in a bounded graph; and applies a guarded consequence rule to a browser/document lineage. The experimental rule requires a mature local baseline, complete provenance, a metadata-only Nyx privacy observation, and a rare descendant egress before it can create an incident. It then records a policy- and authority-gated *future-DNS* playbook in dry-run mode.

The present evidence demonstrates code-local integration, adversarial mechanism tests, and synthetic ingest performance. A 1,000-event local ingest run measured 5,697.04 events/s with p50/p95/p99 synchronous ingest latencies of 0.1740/0.2686/0.3368 ms on the measurement host. These results do not measure browser behavior, DNS socket latency, detection efficacy, false positives, or live enforcement. The work therefore supports a falsifiable next hypothesis rather than a product claim: in an isolated Windows VM, does unified provenance reduce false positives or reveal true-positive consequences that isolated security and privacy signals miss?

1. Problem

Most endpoint decisions begin with an isolated observation: a process starts, a domain is queried, a script is executed, or a data pattern leaves a browser. An isolated observation often lacks the context needed to decide whether it is benign, suspicious, or policy-violating. A browser may legitimately spawn a helper process; a network connection may be routine; a privacy-sensitive field may be sent to a permitted first party. Conversely, a chain of individually weak observations may describe a meaningful consequence.

Valkyrie's research question is not whether a new set of attack signatures can be added to an EDR. It is whether the endpoint can reason locally over a chain such as:

```
interactive application -> descendant process -> new network consequence
+ metadata-only privacy boundary crossing
```

The desired outcome is an explainable, reversible decision at the earliest technically reachable enforcement point, without exporting raw user content to the detection engine.

2. Research Question and Hypothesis

Research question. Can a local provenance graph combine security and privacy observations into a better-supported consequence decision than either observation in isolation?

Current hypothesis. When a complete interactive-process lineage contains both a metadata-only privacy boundary crossing and a rare descendant egress, the combined pattern can justify a more specific, safer response decision than either signal alone.

This is deliberately a limited hypothesis. It does not assume that a privacy event is malicious, that a rare connection is malicious, or that a detected pattern should automatically block a process or domain.

3. System Design

Valkyrie uses a local pipeline:

```
Sensors / Nyx / browser context
-> normalized TelemetryEvent
-> EventBus and EdrEngine
-> CausalityGraph and local baseline
-> detection and consequence scoring
-> policy and authority decision
-> dry-run or explicitly authorized response
```

The normalized schema in [valkyrie/telemetry.py](#) includes process, DNS, network, persistence, malware, asset, and privacy categories. [EdrEngine](#) is the central ingest and correlation point. Its [CausalityGraph](#) models processes using PID plus creation-time identity when available, links parent and child processes, flags inferred or evicted state, bounds memory, and attaches artifacts such as DNS, network, and privacy observations.

Nyx contributes only metadata to this graph. Its outbound observations are normalized into privacy telemetry containing an artifact kind, privacy category, destination host, and first-party origin where available. Request bodies and even masked samples are excluded from graph and incident evidence. This boundary is verified in [tests/test_privacy_consequence.py](#).

The optional Chromium bridge adds browser interaction metadata-navigation, trusted gesture, form submission, and coarse consent-state events-through native messaging and a loopback API. It intentionally does not collect full URLs, query strings, form values, cookies, page content, keystrokes, or DOM snapshots. It also does not fabricate a Windows PID relationship; it remains an unjoined observation layer. The collector and its boundary tests live in [valkyrie/browser_context.py](#) and [tests/test_browser_context.py](#).

4. Experimental Consequence Rule

The narrow rule in `valkyrie/edr/consequence.py` can create a `privacy_consequence` finding only when all of the following are true:

1. The local causal baseline is mature: at least 300 observations across three sessions.
2. The relevant subgraph is complete enough for use: it is not truncated, evicted, or dependent on inferred process nodes.
3. The causal graph's common ancestor is an interactive browser or document application.
4. The chain contains a `nyx_Leak` artifact that holds metadata only.
5. A descendant-not the browser owner itself-has a rare DNS, network, or connection artifact.

The rule explicitly refuses to score malformed or content-bearing privacy metadata, ambiguous privacy destinations, immature baselines, incomplete provenance, non-interactive owners, and common descendant egress. These refusals are central to the experiment: a provenance system should be able to state when its evidence is insufficient rather than silently converting weak context into an enforcement action.

When the rule fires, the EDR records an incident and evaluates the existing policy. Under the standard profile, the tested result is `deceive`; under a stricter profile, a future-DNS block can be considered only through a playbook that requires matching policy and authority decisions. The playbook is dry-run by default. The scorer does not modify threat-intelligence memory and does not claim that the already-observed privacy request was prevented.

5. Method

The current experiment uses three evidence tiers.

5.1 Structural and integration tests

`tests/test_privacy_consequence.py` constructs a browser-owner process, a descendant helper, a DNS artifact, and a metadata-only Nyx artifact. It verifies that the guarded rule fires only under a mature baseline with complete provenance; verifies that raw content and masked samples do not enter the serialized graph; verifies policy escalation behavior; and verifies that a dry-run playbook fails closed without both policy and authority approval.

`tests/test_provenance_adversarial.py` exercises privacy-before-egress event ordering, PID reuse, missing parent provenance, duplicate event identifiers, and a 500-event burst. These are adversarial mechanism checks, not live attack evaluation.

`tests/test_browser_context.py` verifies origin sanitization, input-size and schema rejection, token checks, failure when the token ACL cannot be verified, and loopback API access control. No real browser is required by these tests.

5.2 Safe reproducible demonstration

`tools/provenance_demo.py` executes the actual in-memory Store, EDR, graph, rule, and policy path with synthetic events. It prints a causal chain, the metadata-only evidence, the recorded decision, and a privacy-boundary refusal. It never reads live browser traffic, opens a DNS listener, changes firewall rules, or loads a driver. Its regression test is `tests/test_provenance_demo.py`.

5.3 Synthetic performance measurement

`tools/provenance_benchmark.py` measures the time from entering `EdrEngine.ingest_telemetry()` to return for synthetic DNS artifacts after process nodes are seeded. The benchmark uses a real local Store/EDR/graph pipeline but is intentionally not an end-to-end security test.

6. Results

6.1 Mechanism behavior

The code-local test suite verifies the intended guarded behavior: a mature, complete, metadata-only browser lineage with rare descendant egress produces a `privacy_consequence` incident. The incident has detection, decision, and authority timeline records. The path does not directly mutate intelligence memory and does not execute a network block in the standard demonstration.

The demonstration test additionally verifies that content-bearing metadata is refused with `privacy_boundary_violation`, and that the graph retains neither the provided request body nor the masked sample.

6.2 Synthetic ingest measurement

The recorded 1,000-event run on Windows 11 build 26200 with Python 3.12.10 produced the following local synchronous ingest results:

Measure	Recorded value
DNS artifacts ingested	1,000
Throughput	5,697.04 events/s
Ingest p50	0.1740 ms
Ingest p95	0.2686 ms
Ingest p99	0.3368 ms
Graph nodes / artifacts	2 / 200

The artifact cap was reached by design. The raw local output is retained in [docs/provenance-benchmark-local.json](#).

6.3 Interpretation

The results demonstrate that the selected code path can process synthetic normalized events locally at the measured rate on that host. They do **not** measure DNS socket handling, TLS interception, sensor delay, operating-system scheduling, browser behavior, response execution, false-positive rate, true-positive rate, or live attack detection. They should not be presented as a universal latency value or as proof of real-time enforcement.

7. Limitations and Threats to Validity

The main limitation is attribution. User-mode process polling can miss short-lived processes and can produce incomplete lineage. The current rule refuses incomplete graphs, which protects against overreach but can also cause false negatives. The kernel driver source is not signed, loaded, or validated; therefore Valkyrie does not presently have reliable pre-execution process blocking, pre-write file blocking, or authoritative kernel provenance.

The browser context bridge is also not a process-identity solution. It conveys sanitized context but has no authoritative Windows PID join, cannot establish that a consent dialog was legally meaningful, and cannot enforce a request on its own. TLS inspection is opt-in and cannot inspect certificate-pinned applications. Encrypted DNS can bypass a local DNS control unless separately addressed.

The experiment has no controlled live workload, no benign installer/updater false-positive measurement, no end-to-end observation-to-decision-to-DNS latency measurement, and no live Atomic Red Team validation for this new rule. The available computer is physical hardware rather than a snapshot-capable isolated Windows VM, so running destructive or invasive validation there would not be responsible.

Finally, the system's local-only design reduces external exposure but does not by itself prove privacy, safety, or security. Those properties require independent review and live evidence.

8. Next Experiment

The next experiment should run in a disposable, snapshot-capable Windows VM. It should compare three configurations on identical controlled workloads:

1. Security signals without Nyx privacy artifacts.
2. Privacy signals without provenance-based consequence scoring.
3. The unified, guarded provenance rule.

Each configuration should be evaluated against controlled malicious-like chains and benign browser, installer, and updater workloads. The collection should record attribution completeness, event-to-decision latency, DNS-path latency, detection outcomes, false positives, false negatives, operator reviewability, and rollback behavior. A second VM run after snapshot restore is required before treating a result as repeatable.

Next hypothesis. The unified configuration will either identify a useful class of cross-layer consequences with an acceptable false-positive rate, or it will show that the additional context does not justify its complexity. Both outcomes are valuable because the experiment is designed to be falsifiable.

9. Conclusion

Valkyrie currently demonstrates a local mechanism for unifying privacy and security evidence through process provenance. It has a normalized event layer, a bounded causality graph, metadata-minimizing Nyx integration, a guarded consequence rule, policy and authority checks, adversarial mechanism tests, and a safe synthetic demonstration. The engineering contribution is not an unproven claim that all endpoint consequences can be predicted or blocked. It is the creation of an explicit, testable decision boundary: the system can show what it observed, why it acted, and when it refused to claim enough evidence.

The decisive remaining work is measurement in an isolated Windows VM. Until that evidence exists, Valkyrie should be presented as a promising research prototype with clear implementation evidence and clear empirical limits.

Reproduction

Run the safe synthetic demonstration:

```
python tools/provenance_demo.py
```

Run the focused mechanism tests:

```
python -m pytest -q tests/test_provenance_demo.py tests/test_privacy_consequence.py  
tests/test_provenance_adversarial.py tests/test_browser_context.py
```

Run the synthetic ingest benchmark:

```
python tools/provenance_benchmark.py --events 1000
```

None of these commands changes DNS settings, firewall settings, browser settings, or kernel-driver state. The live-validation program is intentionally not included in this reproduction path.